

1. 论文名称: Retrieval-Augmented Generation **for** AI-Generated Content: A Survey
2. 团队: 北京大学
3. github: <https://github.com/hymiel22/RAG-Survey>

Abstract

- **主题:** 人工智能生成内容 (AIGC) 的发展与挑战
- **关键技术:** 检索增强生成 (RAG)
- **目的:** 解决AIGC面临的挑战, 如知识更新、数据泄露风险和高训练成本
- **方法:** 通过检索过程增强生成器, 提高准确性和鲁棒性
- **内容分类:**
 - **RAG基础:** 根据检索器对生成器的增强方式进行分类
 - **方法论提炼:** 提炼各种检索器和生成器的基本抽象和增强方法
 - **统一视角:** 包含所有RAG场景的综述, 阐明关键技术和方法
 - **额外增强方法:** 总结RAG的工程化和实施方法
- **实际应用:** 调研RAG在不同模式和任务中的应用
- **研究资源:** 介绍RAG的基准测试、局限性和未来研究方向
- **挑战**
 - 知识更新:** 保持知识的最新性
 - 长尾知识:** 处理长尾知识的挑战
 - 数据泄露:** 防止数据泄露的风险
 - 训练成本:** 高昂的训练成本
- **未来研究方向建议**
 - RAG系统的局限性:** 探讨和解决
 - 基准测试:** 提供和改进RAG的评估标准
 - 研究方向:** 为RAG技术的未来研究提供指导

1 Introduction

A. 背景 (Background)

- **AIGC发展**
 - 兴趣激增
 - 多模态内容生成工具发展 (如LLM、DALL-E、Stable Diffusion、Sora)
- **AIGC挑战**
 - 维护最新知识

- 整合长尾知识
- 私有训练数据泄露风险

• RAG介绍

- 解决AIGC挑战
- 适应性数据存储
- 降低生成成本
- 适应多模态应用

• RAG研究缺口

- 缺乏系统性综述
- 文章目的：提供全面的RAG系统综述

• RAG过程与交互

- 检索器定位和查找相关数据源
- 检索结果与生成过程的多种交互方式

• RAG增强与优化

- 提出增强措施改进质量
- 特定组件方法和整个管道优化

• 研究目标

- 提供系统性RAG概述
- 帮助理解RAG在不同领域的应用

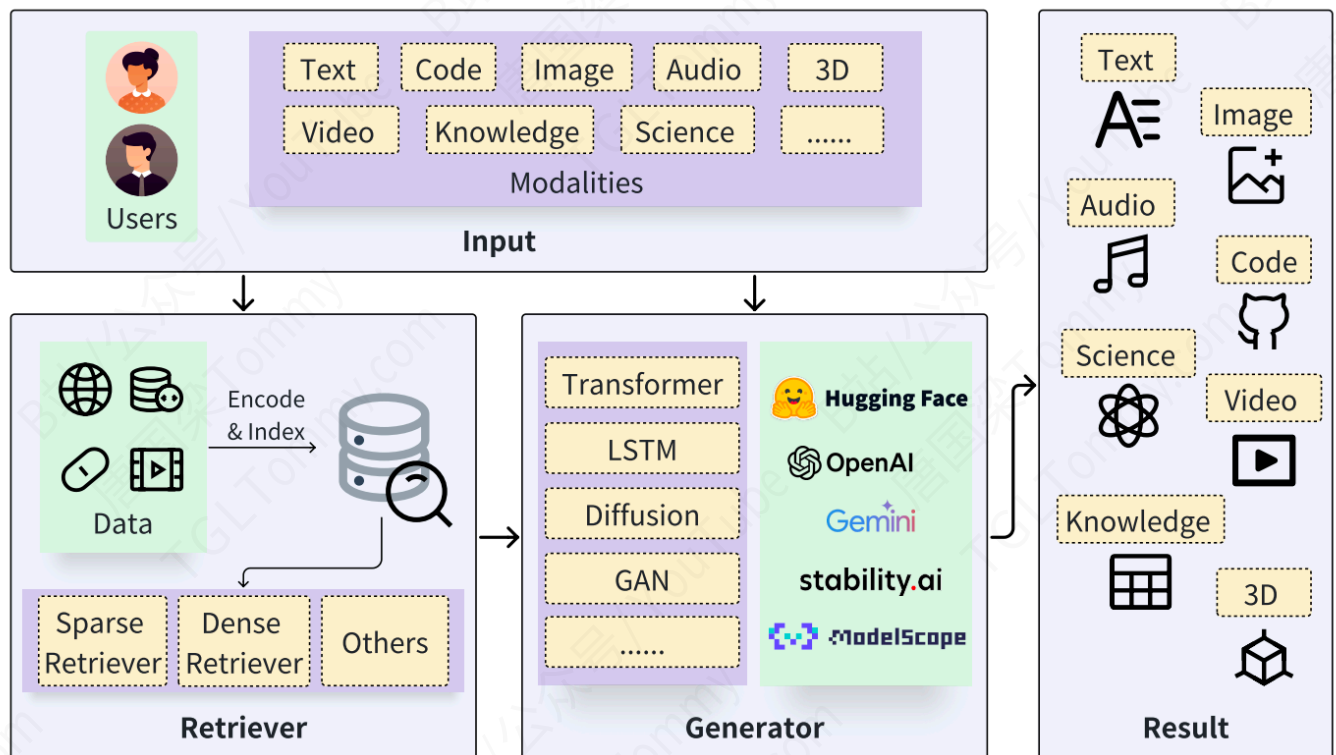


Fig. 1. A generic RAG architecture. The users' queries, which may be different modality, serve as input to both the retriever and the generator. The retriever searches for relevant data sources in storage, while the generator interacts with the retrieval results and ultimately produces results of various modalities.

B. 贡献 (Contribution)

- 综述：提供了一个关于RAG的全面概述，包括基础、增强、应用、基准测试、限制和未来方向。
- **RAG基础**：细化了RAG基础的抽象概念，适用于各种检索器和生成器。
- **RAG增强**：总结了现有RAG流程的增强技术，以实现更有效的RAG系统。
- **AIGC方法**：调查了整合RAG技术的AIGC方法，展示RAG如何促进当前生成模型的发展。
- **限制与未来研究方向**：讨论了RAG的限制和有希望的研究方向，为其未来发展提供了见解。

C. 相关工作 (Related Work)

- 已有调查：
 - Li等人针对文本生成进行了RAG调查。
 - Asai等人提供了关于基于检索的语言模型的教程。
 - Gao等人调查了LLM中的RAG，尤其是基于查询的RAG增强。
 - Zhao等人提出了一个关于多模态RAG应用的调查，但忽略了对RAG基础的讨论。
- 研究缺口：现有调查主要关注LLM促进的文本相关任务，或者只关注单一的RAG基础范式，未全面覆盖RAG的各个方面。
- 本文目标：提供一个系统性的RAG概述，解决现有调查的不足。

2 预备知识

2.1 概述

- **RAG系统组成**
 - 核心模块：检索器与生成器
 - 过程概述：接收查询、信息检索、增强方法传递、内容生成

2.2 生成器

- 功能：核心生成模块
- 性能：超越传统专家系统
- 应用场景：根据任务选择不同模型
- 类型分类：Transformer模型、LSTM、Diffusion Model（扩散模型）、GAN

2.2.1 Transformer

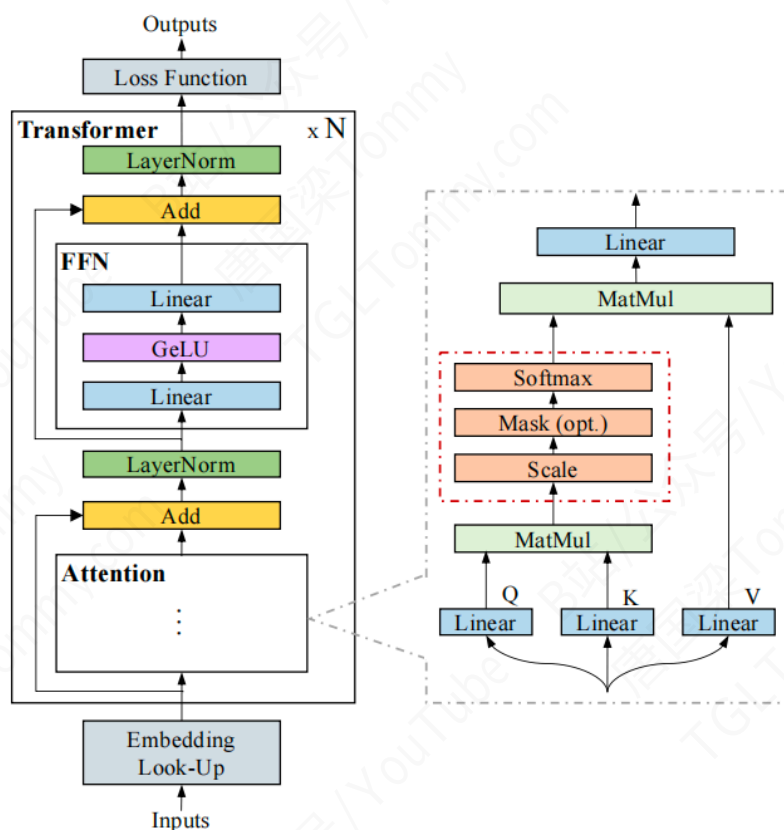


Fig. 2. Architecture of a traditional transformer model.

- 设计目的：解决自然语言处理（NLP）问题。
- 组成部分：
 - 自注意力机制（Self-attention mechanism）
 - 前馈网络（Feed forward network）
 - 层标准化模块（Layer normalization module）
 - 残差网络（Residual network）
- 输入和结构：
 - 输入：Transformer将输入映射为张量 x_{in} ，形状为 (b, s, h) 。
 - b ：批次大小
 - s ：序列长度
 - h ：隐藏维度
- 编码和输出：
 - 位置编码加入到自注意力层与输入张量 x_{in} 。
 - 自注意力模块的输入 x_{in} 与输出 x_{out} 通过残差网络和层标准化模块连接。
 - x_{out} 将被发送到前馈网络。
- 计算过程：
 - $Q = x_{in} * W_q + b_q$

- $K = x_{in} * W_k + b_k$
- $V = x_{in} * W_v + b_v$
- $x_{out} = LayerNorm1(Softmax(\frac{Q * K^T}{\sqrt{h}}) * V * W_o + b_o) + x_{in}$
- $y = LayerNorm2((x_{out} * W_1 + b_1) * W_2 + b_2) + x_{out}$
- 参数：
 - W_q, W_k, W_v, W_o ：可学习张量，形状为 (h, h) 。
 - b_q, b_k, b_v, b_o ：可学习张量，形状为 $(h,)$ 。

2.2.2 长短期记忆网络 (LSTM)

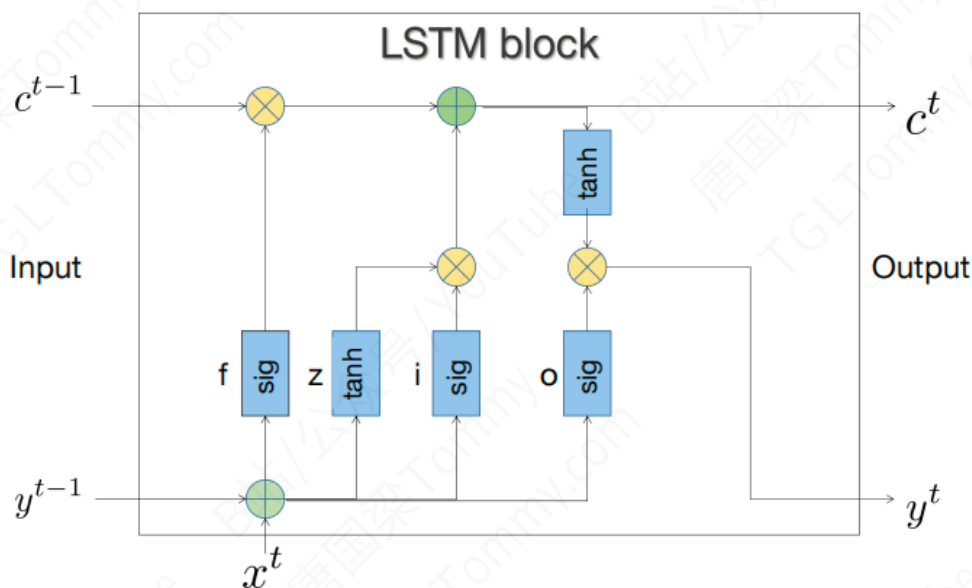


Fig. 3. Architecture of a typical LSTM block with forget gate.

- 特点：是一种特殊的递归神经网络（RNN），解决长期依赖问题。
- 机制：引入细胞状态和门控机制。
- 构成：
 - 输入门（Input Gate）：决定哪些输入数据应保留在cell状态中。
 - 遗忘门（Forget Gate）：确定应丢弃哪些cell状态信息以避免内存过载。
 - 输出门（Output Gate）：决定cell状态中的信息如何影响当前输出。
 - Cell状态（Cell State）：记忆和维持信息的核心模块。

2.2.3 扩散模型 (Diffusion Model)

- **类型**：是一系列可以创建真实和多样化数据样本的深度生成模型。
- **应用**：用于生成图像、文本、视频、分子模型等。
- **原理**：通过逐渐加入噪声使数据变得随机，然后逆转过程从噪声中生成新数据。
- **模型**：主要有三种等效公式：
 - 去噪扩散概率模型 (Denoising Diffusion Probabilistic Models)
 - 基于分数的生成模型 (Score-based Generative Models)
 - 随机微分方程模型 (Stochastic Differential Equations)
- **改进**：包括DDIM、矫正流 (Rectified Flow)、一致性模型 (Consistency Model) 和RPG-DiffusionMaster。

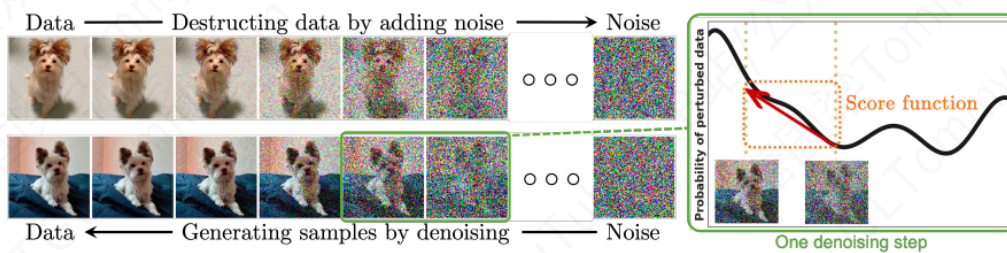


Fig. 4. Diffusion models smoothly perturb data by adding noise, then reverse this process to generate new data from noise. Each denoising step in the reverse process typically requires estimating the score function (see the illustrative figure on the right), which is a gradient pointing to the directions of data with higher likelihood and less noise, *c.f.* [62]

(1) 去噪扩散概率模型 (DDPM)

- **变量定义**：
 - x_0 ：遵循数据分布 $q(x_0)$ 的随机变量。
 - x_t ：在时间步 t 添加噪声后的随机变量，遵循分布 $q(x_t|x_0)$ 。
- **正向过程**：
 - 通过一系列高斯噪声注入扰动数据，将 $q(x_0)$ 转化为简单的先验分布 $q(x_T) \approx N(0, I)$ 。
 - 每个时间步的转换核由 $q(x_t|x_{t-1}) = N(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$ 给出。
- **逆向过程**：
 - 通过逆转正向过程生成新数据样本，使用可学习的马尔可夫链。
 - 先验分布为 $p(x_T) = N(x_T; 0, I)$ ，转换核为 $p_\theta(x_{t-1}|x_t)$ 。
- **模型训练**：
 - 目标是最大化数据 x_0 的变分下界 (VLB)。
 - 通过简化和重参数化技巧，目标函数变为 $E_{t \sim U[1, T], x_0 \sim q(x_0), \epsilon \sim N(0, I)} [\lambda(t) \|\epsilon - \epsilon_\theta(x_t, t)\|^2]$ 。
- **训练目标**：

- 与匹配逆向过程的联合分布 $p_{\theta}(x_0, x_1, \dots, x_T)$ 与正向过程的 $q(x_0, x_1, \dots, x_T)$ 通过最小化KL散度等价。

2.2.4 生成对抗网络（GANs）

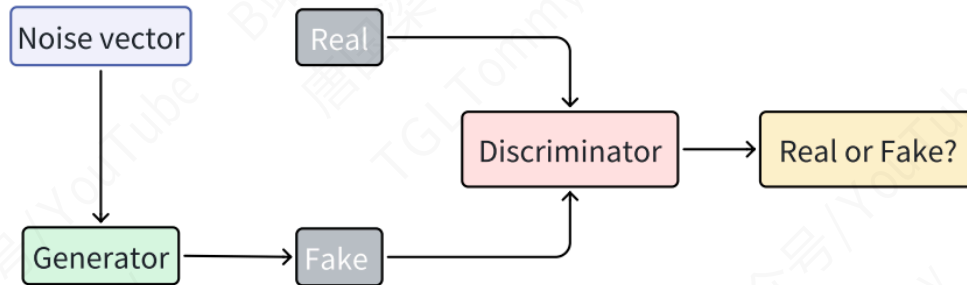


Fig. 5. Architecture of a typical Generative Adversarial Network (GAN).

- **特点：**能够模拟和生成真实感图像、音频等数据的深度学习模型。
- **组成：**
 - 生成器（Generator）：生成逼真样本。
 - 鉴别器（Discriminator）：鉴别真伪样本。
- **工作方式：**两部分通过对抗学习竞争，互相提升性能。

2.3 检索器

- **功能：**识别和获取与信息需求相关的信息系统资源。
- **检索过程：**
 - 将信息资源概念化为键值对 $\{(k_i, v_i)\}_{i=1}^N$ 。
 - 使用相似性函数 s 搜索最相似的键，并获取配对的值。
- **检索方法分类：**基于不同相似性函数，可分为稀疏检索和密集检索等。

2.3.1 稀疏检索器

- **应用：**广泛用于文档检索。
- **方法：**
 - TF-IDF：词频-逆文档频率。
 - 查询可能性。
 - BM25：基于词统计分析构建倒排索引的方法。
- **BM25计算：**
 - 对于包含关键词 $\{q_i\}_{i=1}^n$ 的查询 q ，文档 D 的BM25得分为：

$$s(q, D) = \sum_{i=1}^n IDF(q_i) \cdot \frac{f(q_i, D) \cdot (a+1)}{f(q_i, D) + a \cdot (1-b+b \cdot \frac{|D|}{avgdl})}$$

- $IDF(q_i)$ 为逆文档频率权重，计算为：

$$IDF(q_i) = \ln \left(\frac{N-n(q_i)+0.5}{n(q_i)+0.5} \right)$$

- 参数定义：

- $f(q_i, D)$ ：关键词 q_i 在文档 D 中出现的次数。
- $|D|$ ：文档 D 的长度。
- $avgdl$ ：语料库中文档的平均长度。
- a 、 b ：可调参数。

2.3.2 密集检索器

- 特点：与稀疏检索不同，使用密集嵌入向量来表示查询和键，并建立近似最近邻（ANN）索引以加快搜索。
- 技术实现：
 - 利用BERT、RoBERTa等预训练模型作为编码器独立编码查询和键。
 - 用于文本、代码、音频、图像和视频数据的模型编码。
- 相似性计算：使用余弦、内积、L2距离等方法计算密集表示之间的相似性。
- 训练方法：通常遵循对比学习范式，使正样本更相似，负样本更不相似，并采用硬负样本技术进一步提高模型质量。
- 推理过程：使用近似最近邻方法进行高效搜索，开发了多种索引来支持ANN搜索，如树索引、局部敏感哈希、邻居图索引等。

其他检索方法

- 备选方法：
 - 计算自然语言文本或代码片段抽象语法树（AST）的编辑距离。
 - 知识图谱中实体与关系的预构建索引检索。
 - 使用RAG方法，通过跳转检索来利用多跳的知识图。
 - 命名实体识别（NER）作为检索，输入是查询，实体是键。

3 方法

3.1 RAG基础

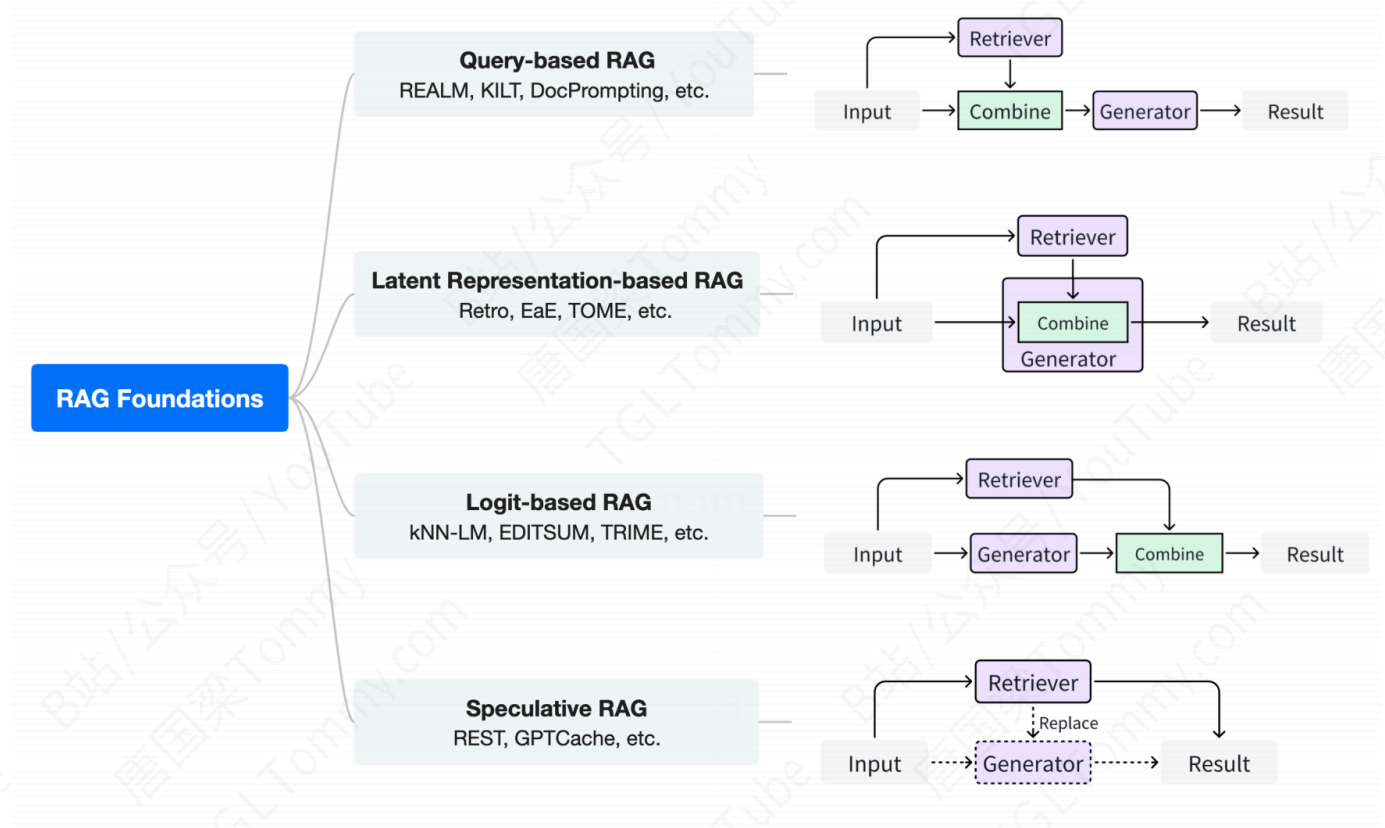


Fig. 6. Taxonomy of RAG foundations.

3.1.1 Query-based RAG

- **查询基础RAG**：结合用户查询和检索过程中获取的文档信息。
- **RAG实例**：
 - REALM：使用双BERT框架来实现知识检索和整合。
 - RAG：结合预训练语言模型和知识检索机制。
 - RAG-Token 和 RAG-Sequence：两种检索信息的RAG变体。
 - KILT：关注信息精确性和可靠性。
 - SELF-RAG：通过检索和评估策略来增强回答的准确性和相关性。
 - REPLUG：不修改语言模型架构而实现RAG的方法。
- **应用增强方法**：
 - **代码生成**：
 - APICoder 和 DocPrompting：展示检索信息整合到语言模型中如何提高代码生成的准确性和相关性。
 - CEDAR 和 InferFix：在自动程序修复任务中利用检索的代码段。
 - ReACC：在代码补全任务中使用检索的代码段作为输入的新部分。
 - **音频生成**：
 - MakeAnAudio：利用检索来构建无语音音频的字幕，以减少文本到音频训练的数据稀疏性。
 - **知识库问答 (KBQA)**：
 - Uni-Parser、RNG-KBQA 和 ECBRF：通过结合检索和语言模型来提高QA系统的性能和准确性。

- **科学领域：**
 - GeneGPT：用于基因组学查询。
 - ChatOrthopedist：旨在帮助青少年进行脊柱侧弯的共同决策。
- **图像生成：**
 - RetrieveGAN：通过整合检索的信息来提高生成图像的相关性和准确性。
- **3D生成：**
 - RetDream：在3D生成领域，利用CLIP检索相关的3D资产，并与用户输入合并。

3.1.2 Latent Representation-base RAG

1. 潜在表示基础的RAG：

- 生成模型与检索对象的潜在表示互动，从而提高理解能力和生成内容的质量。

2. FiD (Fusion-in-Decoder) 技术：

- 利用BM25和DPR检索支持段落。
- 将每个检索段落及其标题与查询连接，通过编码器单独处理。
- 通过在解码器中融合来自多个检索段落的信息来生成答案，而不是在编码器中处理每个段落。
- FiD方法已广泛应用于知识库问答 (KBQA)，通过Fusion-in-Decoder技术显著提高问答系统的性能。

3. Retro技术：

- 通过“Chunked Cross-Attention”机制将输入序列分成独立的块，每个块独立执行跨注意力操作，减轻计算负担。
- 使模型能够为不同的序列段检索和同化不同文档，支持整个生成过程的动态检索。

4. TOME：

- 重点是实体提及的压缩编码，存储大约1.5亿个实体的编码。
- 通过实体ID，允许检索更细粒度的信息。

5. 记忆型Transformer (Memorizing Transformers)：

- 通过kNN-augmented attention机制集成到Transformer层，以实现输入序列处理中的kNN搜索。
- 无需完全重新训练即可提升性能，还扩大了模型的记忆范围。

6. EaE (Entity-as-Experts)：

- 使语言模型内化明确的实体知识。
- 通过实体特定的参数化和实体内存层优化推理效率。

7. 多模态生成技术：

- 如ReMoDiffuse和AMD在3D生成领域中，使用文本描述来提高生成相应3D动作的准确性。
- 在音频领域，如AudioLDM整合文本和音频特征。
- 在视频字幕领域，如R-ConvED和CARE采用了注意力机制来生成隐藏状态，并进而生成字幕。

3.1.3 Logit-based RAG

1. Logit-based RAG:

- 在解码过程中，生成模型通过logits结合检索信息。
- 通常，logits会被求和或者通过模型结合来产生逐步生成的概率。

2. kNN-LM模型:

- 将预训练的神经语言模型与k最近邻搜索结合。
- 使用预训练模型生成候选词列表，通过检索数据仓库中与当前上下文最相关的k个邻居来提高原始语言模型的输出。
- 模型核心在于能够动态检索广泛文本语料库的信息，特别是在处理罕见模式和适应不同领域时，显著提高预测的准确性和相关性。

3. TRIME:

- 在训练和测试阶段实现记忆合并，将新数据批次和记忆构建技术结合起来有效利用外部记忆。
- 使用BM25分数将具有高词汇重叠的段落打包到同一个批次中，构建训练记忆以进一步优化模型性能。

4. NPM模型:

- 一种非参数化遮蔽语言模型，由编码器和参考语料库组成。
- 编码器的作用是将语料库中的短语映射为固定大小的向量，并通过检索与遮蔽位置最相似的短语来填充[MASK]。

3.1.4 Speculative RAG

- 寻找使用检索替代生成的机会以节省资源和加快响应速度。
- REST[30]用检索替换推测性解码中的小模型来生成草稿。
- GPTCache[37]尝试通过建立语义缓存来解决使用LLMs API时高延迟的问题，以存储LLM的响应。

3.2 RAG 增强

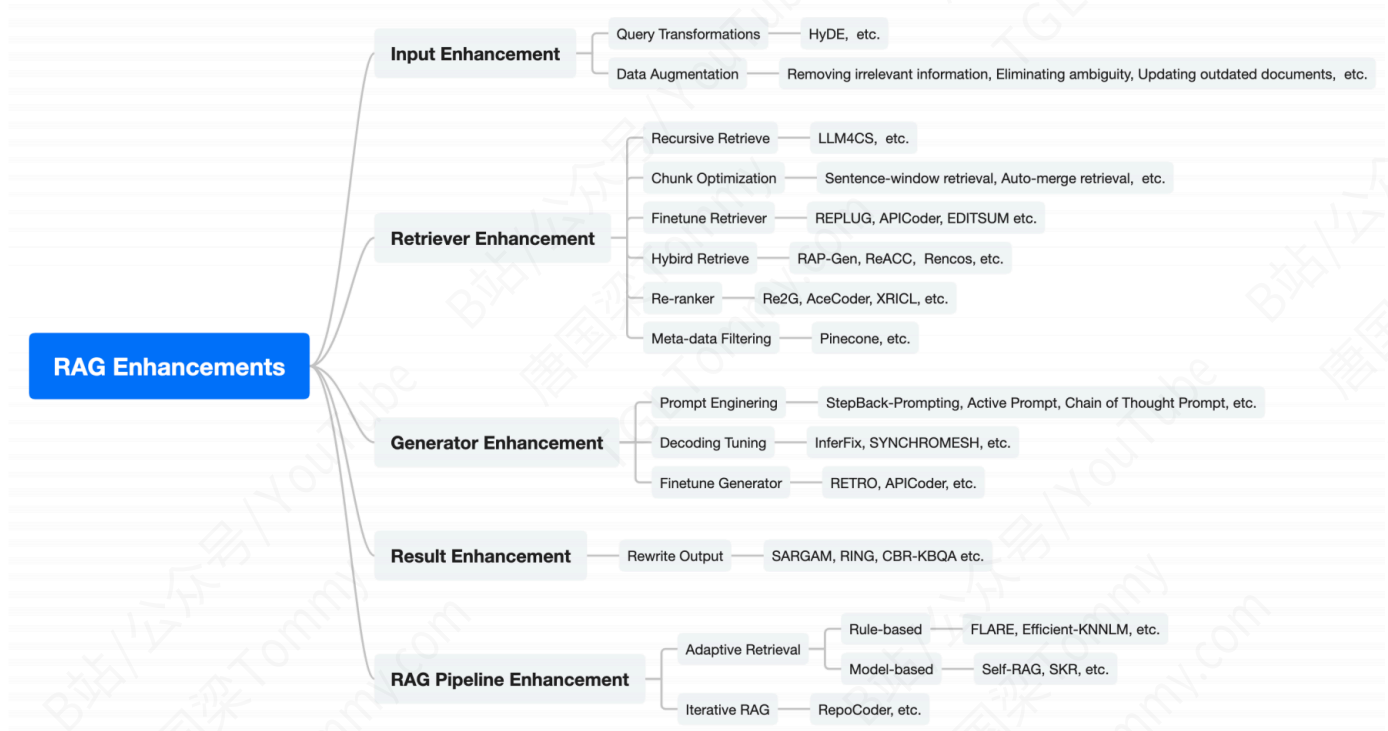


Fig. 7. Taxonomy of RAG Enhancements.

3.2.1 输入增强（Input Enhancement）

- **查询转换（Query Transformation）**：通过修改输入查询，使用Query2doc和HyDE首先用查询生成一个伪文档，然后使用这个文档作为检索的关键。
- **数据增强（Data Augmentation）**：指在检索前提前改善数据，例如去除无关信息、消除歧义、更新过时的文档，合成新数据等。

3.2.2 检索器增强（Retriever Enhancement）

- **递归检索（Recursive Retrieve）**：分割查询，并进行多次搜索以检索更多、质量更高的内容。
- **块优化（Chunk Optimization）**：调整块的大小以获得更好的检索结果。
- **微调检索器（Finetune Retriever）**：通过高质量的域数据或任务相关数据对已经具有良好表达能力的嵌入模型进行微调。
- **混合检索（Hybrid Retrieve）**：同时使用多种检索方法。
- **重排序（Re-ranking）**：重新排序检索内容以获得更大的多样性和更好的结果。
- **元数据过滤（Meta-data Filtering）**：使用元数据（如时间、目的等）过滤检索到的文档，以获得更好的结果。

3.2.3 生成器增强 (Generator Enhancement)

在RAG系统中，生成器的质量常常决定了最终输出结果的质量，因此，生成器的能力决定了整个RAG系统有效性的上限。以下是对生成器增强的几种方法的概述：

1. 提示工程 (Prompt Engineering)：

- 专注于提高语言模型输出质量的技术，如提示压缩、Stepback提示、活跃提示、思维链提示等。
- 这些技术适用于RAG系统中的大型语言模型生成器，可以通过压缩查询来加快模型推理速度，减轻不相关信息对模型的负面影响，缓解“中途迷失”现象。

2. 解码调整 (Decoding Tuning)：

- 在生成器处理过程中添加额外控制，如调整超参数以实现更大的多样性，限制输出词汇等。
- InferFix调整解码器的温度以平衡多样性和结果质量，SYNCHROMESH限制输出词汇以减少执行错误，DeepICL根据额外的温度因素控制生成的随机性。

3. 微调生成器 (Finetune Generator)：

- 通过微调生成器来增强模型的领域知识或与检索器更好的匹配度。
- 例如，RETRO修复检索器的参数并在生成器中使用块状交叉注意力机制结合查询和检索内容，API Coder微调生成器以适应新的API信息和代码块，CARE先冻结编码器和检索器然后微调解码器以减少字幕损失和概念检测损失。

3.2.4 结果增强 (Result Enhancement)

结果重写 (Rewrite Output)：

- **SARGAM**：通过删除分类器、占位符分类器和插入分类器来修改代码相关任务中生成的结果，并使用Levenshtein Transformer更好地适应实际的代码上下文。
- **Ring**：通过重新排序基于每个token的对数概率平均值的候选者来获得多样性结果。
- **CBR-KBQA**：通过将生成的关系与知识图谱中查询实体的局部邻域中呈现的关系对齐，来修正结果。

3.2.5 RAG流程增强 (RAG Pipeline Enhancement)

• 自适应检索 (Adaptive Retrieval)：

- **规则基础 (Rule-based)**：通过概率判定是否进行搜索。
- **模型基础 (Model-based)**：Self-RAG使用训练过的生成器来决定是否执行检索，并通过Self-Reflection token评估最终输出结果的质量。

• 迭代RAG (Iterative RAG)：

- **RepoCoder**：应用迭代检索-生成循环来增强代码补全任务中的检索查询，并在每一次迭代中得到更好的结果。
- **ITER-RETGEN**：在迭代过程中，生成器当前的输出可以在某种程度上反映其仍然缺乏的知识，因此迭代检索能反映出下一轮应该获取的信息，从而提高下一轮生成内容的质量。

4 应用

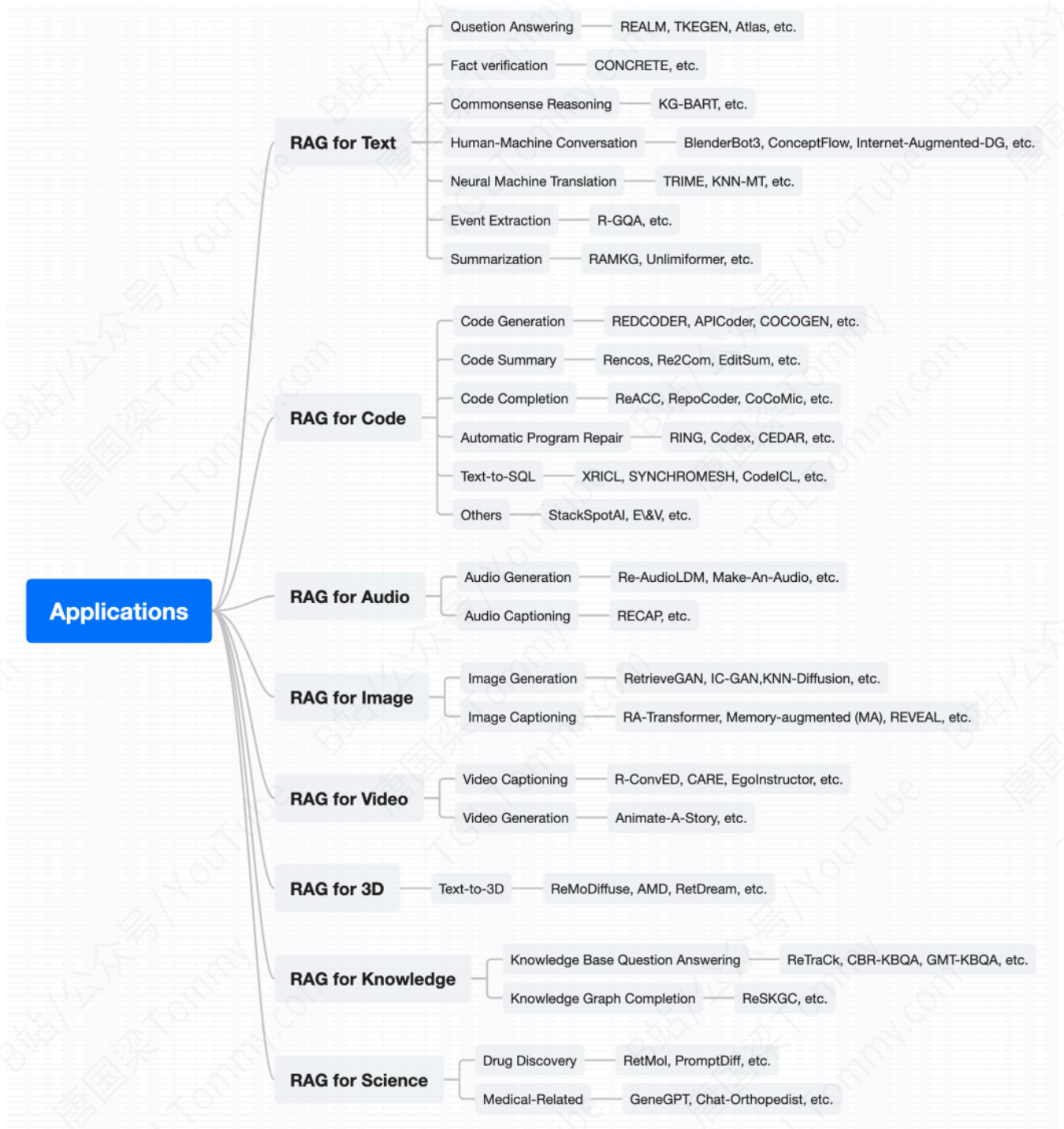


Fig. 8. Taxonomy of multiple applications in multimodality.

A. RAG for text

1. 问题回答：问题回答涉及通过检索大量问题相关文档并生成回答的过程。

- FID和REALM用于识别与查询最相关的文章片段，并将每个片段与问题相结合以生成回答。
- TOTANOVA等人使用知识图谱中的子图替换REALM中的文本语料。

- Atlas提出利用RAG帮助LLMs（大型语言模型）在开放域问答任务中显著提升少样本学习能力。
- RETRO使用注意力机制在模型内整合检索到的相关文档，以生成最终答案。
- SKR观察到使用RAG可以通过引导模型评估其对相关知识的掌握来提高答案质量，并随后适应其对检索资源的使用。
- KAPING通过构建提示将知识图谱输入到LLM以提高性能。
- TOG介绍了一个创新的知识增强LLM框架，该框架通过扩展LLMs和知识图谱的交互以及使用波束搜索来扩展推理路径空间来解决长尾问题。
- NPM利用非参数数据分布代替softmax层，使得模型能够更有效地执行。
- Self-RAG通过学习何时改进答案质量，并通过反思性标记来审查最终生成的结果。
- CL-RelKT使用语言泛化编码器来弥合问题-文档对之间的差距。
- CORES利用跨语言检索指南，通过引入一个新的密集型检索算法和多语言自适应生成模型LALY来减少语言资源不平衡。
- EAE检索实体嵌入以提高查询质量，并将这些与隐藏状态结合以进一步处理。
- UR-QA在遇到未见过问题时通过比较校准的置信度来改进答案。

2. 事实验证：事实验证涉及评估信息真实性，这在NLP、信息检索和数据挖掘等学科中是关键功能。

- CONCRETE利用跨语言检索机制来利用多语言证据，有效地弥合在事实检查数据集中语言资源不足的差距。

3. 常识推理：机器在人类方式上推理或决策的能力称为常识推理，这在NLP领域是一个持续的挑战。KG-BART通过在知识图谱中纳入复杂的概念互相关系，扩展了概念性知识的范围。

4. 人机交互对话：人机交互对话涉及机器理解人类语言并利用该技能与人类无缝交互的能力。ConceptFlow通过使用常识知识图谱来结构化对话，引导对话的流向。BlenderBot3则扩大了其搜索视野，不仅挖掘互联网内容，还考虑历史对话记录以改善对话生成质量。

5. 神经机器翻译：自动化文本翻译是NLP和AI领域的一个重要目标。例如，Cai等人提出了一种新颖的方法，结合单语语料库和多语言学习技术，挑战传统的神经机器翻译中依赖双语语料库的做法。

6. 事件提取：事件提取专注于从非结构化文本数据中识别和提取特定事件类型。R-GQA使用基于检索的方法来增强问题回答的上下文。

7. 摘要：摘要是NLP领域的一项任务，旨在从长篇文本中提取关键信息并生成简洁的摘要。有两种主要方法：提取式摘要和抽象式摘要。RAMKGP和Unilmformer等工具在提高关键短语生成和处理长输入方面取得了进展。

B. RAG for Code

1) 代码生成 (Code Generation)

- **目的：**将自然语言描述 (NL) 转换为代码实现。
- **技术：**使用序列到序列的模型如LSTM、Transformers和仅解码器的大型语言模型 (LLMs)。
- **检索方法：**通过抽象语法树 (AST) 或文本编辑距离来找到相似的代码片段。
- **研究实例：**
 - **RECODER：**利用密集检索器检索类似的NL描述，然后串联NL文本和它们配对的代码，为下游生成器PLBART提供输入。
 - **APICoder：**通过对齐NL描述和API文档的嵌入来训练基于Bert的深度检索器，然后检索相关API信息为生成器建立提示。
 - **COGEGEN-MONO：**旨在进行常识推理，生成基于代码的推理图。
 - **RECODE：**使用编辑距离检索类似的NL描述，然后提取n-gram动作子树。

2) 代码总结 (Code Summary)

- **目的：**将代码转换为自然语言描述。
- **技术：**同样使用序列到序列的模型作为生成器。
- **检索方法：**利用额外的编码器处理检索结果，比如抽象语法树 (AST) 的语法相似性和稠密表示之间的余弦相似性。
- **研究实例：**
 - **Rencos：**使用两种不同的方法检索类似的代码片段。
 - **Re2Com、EditSum：**使用稀疏检索方法和基于LSTM的生成模型。

3) 代码自动完成 (Code Completion)

- **定义：**相当于编码中的“下一句预测”任务。
- **技术：**遵循少数示例学习范式，结合稀疏和密集检索的混合检索器，以及适应CodeGPT的生成器。
- **研究实例：**
 - **ReACC：**遵循少数示例学习范式，结合稀疏和密集检索的混合检索器，以及适应CodeGPT的生成器。
 - **RepoCoder：**进一步进行迭代检索和生成，以缩小检索上下文和预期完成目标之间的差距。

4) 自动程序修复 (Automatic Program Repair)

- 自动程序修复是利用生成模型自动输出正确版本代码的过程。这个过程可以大大节约修复代码的努力。
- RING[190]通过稀疏和密集向量检索出相似的错误信息，然后为生成器Codex构建提示。
- CEDAR[130]用于断言生成和程序修复任务，它使用稀疏和密集检索来搜索相似的代码，然后为Codex生成提示。

- InferFix[131]构建了一个包含bug类型、位置、相关语法层次结构和类似修复的提示，然后使用Codex进行修复。
- RAP-Gen[174]通过包括稀疏和密集检索器的混合检索器改进了这个过程，并对CodeT5[105]使用RAG范式进行微调。
- SARAGAM[189]使用密集检索来搜索类似的错误代码，然后应用另一个模型来修改最终结果。

5) 文本-to-SQL和基于语义解析的代码生成 (Text-to-SQL and Code-based Semantic Parsing)

- 语义解析是将自然语言陈述转换为结构化含义表示的过程，其中代码语言经常被用来增强这一过程。
- SQL不仅是一种编程语言，也可以被视为结构化表示，因此在文本到SQL的过程中将其放在特殊情况下进行讨论。
- XRCL[178]关注将非英语陈述翻译成SQL查询的问题，它通过密集检索搜索和重排序英语陈述，并为Codex构建生成SQL查询的提示。
- SYNCROMESH[200]提出约束语义解码以在SQL或其他语言的生成过程中强化丰富的语法和语义约束。
- CodeICL[225]使用Python进行语义解析，并用结构化领域描述来增强GPT生成的提示。

6) 其他 (Others)

- 有几种其他代码相关任务采用了RAG范式。
- E&V[227]使用基于LLM的代理进行程序静态分析。
- StackSpotAI[228]构建了一个AI编程助手，整合了许多代码相关任务。

C. RAG for Audio

1) 音频生成 (Audio Generation)

- **目标:** 通过自然语言输入生成音频。
- **Re-AudioLDM[157]:** 采用检索器CLAP[25]检索相似的标题-音频对，并将这些对作为输入。
- **生成模型:** 使用生成器、潜在扩散模型和基于VAE的解码器。
- **Make-An-Audio[42]:** 使用稠密检索器CLAP[25]增强数据，检索与自然语言文本相关的音频，然后构建用于扩散基础文本到音频模型训练的伪提示。

2) 音频字幕 (Audio Captioning)

- **目标:** 将音频数据与自然语言数据结合生成自然语言描述，本质上是序列到序列的任务。
- **RECAP[229]:** 使用CLAP[25]检索器检索与给定音频数据相关的标题。
- **GPT-2模型:** 利用检索到的标题构建提示输入，通过交叉注意力与音频嵌入互动。
- **研究方法[41]:** 使用检索器VGGish[106]产生音频的密集嵌入，和GPT-2结合生成检索标题的表示，这些标题与相似音频配对。
- **融合信息:** 通过一个额外的多头注意力模块和线性层融合音频和标题信息，生成输出。
- **LLMs研究[230]-[232]:** 利用深度检索模型，将不同模式对齐到相同的潜在空间以生成下游文本。

D. RAG for Image

1) 图像生成 (Image Generation)

- RAG通过整合信息检索系统来增强生成模型。对于图像生成，意味着利用检索来的数据产生高保真度和真实感的图像。
- RetrieveGAN利用Gumbel-softmax技巧进行检索，以选取合适的图像块作为生成参考，同时优化嵌入功能。
- IC-GAN可以生成真实且多样的图像，并能通过改变检索条件和生成图像的类型来转移到未见过的数据集。
- KNN-Diffusion不使用文本数据训练扩散模型，而是基于CLIP和kNN嵌入生成图像。
 - RDM结合了扩散或自回归模型与大型外部图像数据库，通过检索邻近图像来改善训练。
 - Re-Imagen通过检索信息来生成真实感的图像。
 - X&Fuse是一种通用方法，通过将条件图像与噪声图像串联，允许在U-Net架构中的每个注意力块之间进行交互。

2) 图像字幕 (Image Captioning)

- 采用检索增强型图像字幕方法，不同于传统方法，它通过检索一组数据字幕来生成描述性句子。
- Memory-augmented (MA)从记忆库中提取显式知识来改进字幕生成。
- RAMP通过对抗性训练整合检索到的候选字幕来增强图像段落字幕。
 - RA-Transformer通过kNN记忆体来增强生成过程，达到了COCO数据集的竞争性表现。
 - EXTRA通过从相似图像检索字幕来增强图像字幕，显示出检索足够数量的字幕（例如k=5）可以提高视觉质量。
 - REVEAL是一个检索增强型模型，利用世界知识和多模态来源来学习。
 - SMALLCAP是第一个利用检索增强技术进行域传输和概括化的图像字幕模型。
 - Cross-Modal Retrieval and Semantic Refinement (CSR)方法通过语义精炼来加强跨模态检索，产生准确且富含上下文的字幕。

E. RAG for Video

1) Video Captioning

- 视频字幕是用描述性语言描述视频内容。
- **R-ConvED** 使用双编码进行视频-文本检索，并提出了一个卷积编解码器网络来生成字幕。
- **CARLE** 利用帧、音频和检索文本的编码器来预测目标字幕。
- **EgoInstructor** 生成第一人称视角视频的字幕，通过检索相关的视频和文本，并通过基于转换器的解码器与编码视频进行交互。

2) Video Generation

- 文本到视频的生成是根据文本描述生成视频。
- **Animate-A-Story** 根据文本生成故事讲述视频。
- **Diffusion model** 通过扩散模型生成两个分支：文本编码器和结构编码器，来控制检索视频的结构。

F. RAG for 3D

1) Text-to-3D

- **ReMoDiffuse [49]**: 利用扩散模型从文本生成3D运动实体。先通过CLIP检索相关实体，然后通过语义调制的注意力层使用这些信息。
- **AMD [156]**: 设计了两个运动扩散分支，用于保真度和多样性。第一个分支输入原始文本用于扩散，第二个分支将文本解构为解剖脚本并检索类似参考运动。使用变换器基础融合模块来适应性地平衡两个分支的结果。
- **RetDream [48]**: 使用检索到的3D资产来增强从2D扩散模型到3D生成的变分分数蒸馏过程。使用CLIP检索3D资产，并利用这些资产提供几何先验和2D先验。

G. RAG for Knowledge

1) Knowledge Base Question Answering

- 利用知识库和知识图谱结构化知识进行问题回答。
- **ReTraCK [239]**、**Unseen Entity Handling [51]**、**CBR-KBQA [52]**、**GMT-KBQA [50]**、**RNG-KBQA [121]**、**TIARA [122]** 和 **UniK-QA [142]** 等模型通过检索相关实体和关系，生成逻辑形式答案或SPARQL输出。
- **EBRBF [134]**、**DECAF [143]**、**KD-CoT [145]** 和 **FC-KBQA [243]** 等使用不同的检索和融合技术来生成基于知识的回答。

2) Knowledge Graph Completion

- 目的是预测知识图谱中缺失的实体或关系。
- **ReSKGC [146]** 将所有训练三元组线性化为文本，然后使用BM25检索相关三元组并通过T5与融合-解码器生成完整三元组。

H. RAG for Science

1) Drug Discovery

- **RetMol [53]**: 结合了一种轻量级检索机制和一个预训练的编码器-解码器生成模型，使用SMILES字符串表示分子，并与ChemFormer模型一起，实现分子的检索和融合。
- **PromptDiff [250]**: 为结构基础药物设计（SBDD）而设计，使用3D扩散模型促进目标感知的分子生成，检索优化了配体参考以指导扩散模型合成满足特定设计标准的配体。
- **DeepICL [147]**: 为基于交互的3D分子生成创建了一个架构，侧重于蛋白质-配体相互作用的建模，旨在促进可概括性强的口袋受限配体设计。

2) Medical Applications

- **GeneGPT [54]**: 是一种新方法，通过利用国家生物技术信息中心（NCBI）提供的Web API来训练LLM，从而提升检索医学领域特定数据库的能力。
- **Chat-Orthopedist [136]**: 构建了一个关于青少年特发性脊柱侧凸（AIS）疾病和治疗选项的外部知识库，并开发了一个检索增强型ChatGPT来提供准确和易于理解的响应。

5 基准测试

5.1 Chen et al. [252] 提出的RAG基准测试

- **噪声鲁棒性 (Noise Robustness)**: 评估LLMs能否从包含噪声的文档中提取所需信息。
- **负面拒绝 (Negative Rejection)**: 评估LLMs在检索内容不足时是否会拒绝回答查询。
- **信息整合 (Information Integration)**: 评估LLMs是否能够获得知识并通过整合多个检索内容来做出回应。
- **反事实鲁棒性 (Counterfactual Robustness)**: 评估LLMs识别检索内容中反事实错误的能力。

5.2 其他三个基准测试

- **RAGAS [253], ARES [254] 和 TruLens [255]** 考虑了三个不同方面：
 - **真实性 (Faithfulness)**: 关注结果中的事实错误。

- **答案相关性 (Answer Relevance):** 测量生成结果是否实际解决了问题（即查询）。
- **上下文相关性 (Context Relevance):** 判断检索内容是否包含尽可能多的知识来回答查询，并且包含尽可能少的不相关信息。

5.3 CRUD-RAG [256]

将所有RAG任务分为四个类别：创建（Create）、读取（Read）、更新（Update）和删除（Delete），并使用文本续写、单文档和多文档的问题回答、幻觉修改和开放域多文档摘要等评估每个类别。

6 讨论

A. 限制

1. **检索结果的噪音:**
 - 信息检索可能因信息丢失而无法产生完美结果。
 - 检索产生的噪声可能有助于增加生成过程中的多样性。
2. **额外开销:**
 - 检索过程可能引入额外的计算开销。
 - 递归检索等复杂方法会增加更多开销。
3. **检索与生成的交互:**
 - 实现检索和生成之间的无缝交互需要精心设计和优化。
 - 目前还需要进一步的研究来优化这两个组件的交互。
4. **长上下文生成:**
 - 长上下文生成对于RAG系统的优化是一个挑战。
 - 近期研究表明，长上下文模型可能不会完全取代RAG。

B. 潜在的未来方向

1. **RAG方法论、增强和应用的进一步研究:**
 - 探索检索者和生成器之间不同交互模式的更高级方法论。
 - 对RAG的增强有助于提高特定组件或整个流程的有效性。
2. **高效部署与处理:**
 - 当前存在的RAG解决方案缺乏即插即用的解决方案。
 - 实现RAG的高效处理仍是一个挑战。
3. **融入长尾知识和实时知识:**
 - 利用实时和长尾知识是RAG的主要动机之一。
 - 设计一个能够持续更新知识的有用RAG系统是一个不断增长的研究方向。

4. 与其他技术结合:

- RAG与其他旨在提高AIGC有效性的技术正交。
- 进一步研究如何结合这些技术以充分利用它们的潜力。

7 结论

- **调查重点：**文中对与AIGC相关的RAG应用场景进行了全面调查，重点关注了增强基础、增强手段和实际应用。
- **基础范式总结：**系统地组织了RAG的基本范式，提供了检索器与生成器交互的深入理解。
- **RAG系统构建与评估：**回顾了基于构建的RAG系统的增强手段，旨在提升输入、检索器、生成器的效能和最终生成结果的质量。
- **实际应用展示：**展示了RAG在多种模态和任务中的应用实例，为研究人员提供参考。
- **基准测试与未来方向：**介绍了RAG的基准测试，讨论了当前的限制，并指出了未来研究的可能方向。